A C-style cast quietly picks a cast for you. C++ makes you name it.

```
Derived* d = (Derived*)b;
```

Which cast did the compiler choose here? You cannot tell by reading it.

swipe →

The four casts

CAST	USE IT TO	CHECKED
`static_cast`	convert normally	at compile time
`dynamic_cast`	downcast safely	**at run time**
`const_cast`	add or remove `const`	at compile time
`reinterpret_cast`	reread the bits	**not at all**

Reach for them **in this order**. The further down, the rarer it should be.

static_cast

```
double d = 3.9;
int i = static_cast<int>(d);   // 3

Base* b = new Derived;
Derived* p = static_cast<Derived*>(b);
```

The everyday cast. Numbers, and pointers up or down a class hierarchy.

Downcasting is a promise.

If `b` is not really a `Derived`, the behaviour is undefined. Nothing checks it.

dynamic_cast

```
if (auto* p =
        dynamic_cast<Derived*>(b)) {
    // b really is a Derived
}
```

The same downcast, but **checked at run time**.

- a pointer gives `nullptr` if it is wrong
- a reference throws `std::bad_cast`
- needs a `virtual` function in `Base`

It costs a lookup. If you use it a lot, a virtual function is usually the better design.

The two you rarely need

const_cast

```
legacy(const_cast<char*>(s));
```

For an old function that takes `char*` but only reads. Writing to an object that was declared `const` is undefined behaviour.

reinterpret_cast

```
auto* bytes =
    reinterpret_cast<const unsigned char*>(&x);
```

Reading an object as bytes is allowed. Reading it as another type is not. For that, use `memcpy` or `std::bit_cast`.

Which one do I use?

YOU WANT TO	USE
convert a number or a type	`static_cast`
downcast and you are not sure	`dynamic_cast`
drop `const` for an old API	`const_cast`
look at raw bytes	`reinterpret_cast`

And avoid the C-style cast: it silently becomes a `static_cast`, `const_cast` or `reinterpret_cast`, **whichever compiles**.

How many C-style casts are in your codebase?